Программная инженерия

Лекция 10: Monitoring и Observability

Денисов Артём Владимирович

Уральский Федеральный Университет

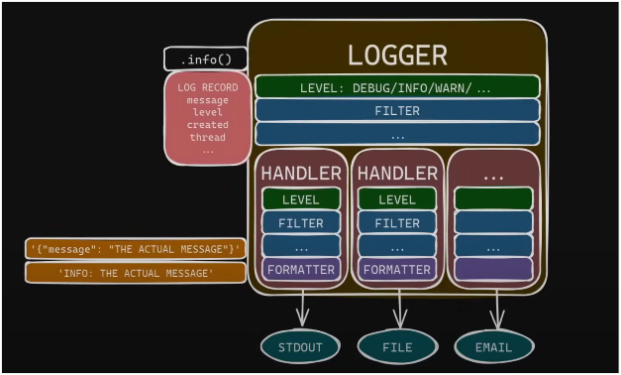
2026

Проблема

Логи в крупных проектах разделены

- Один пользовательский запрос проходит через несколько сервисов
- У каждого сервиса – свои логи и свой контекст
- Ошибка видна сразу в нескольких местах

У каждого сервиса свой агент для логирования



Логгеры в разных сервисах работают по-разному

Решение?

Сохранять логи в один log файл или в общую директорию

- Единая точка поиска
- Синхронизировать логи разных сервисов по времени (общий TimeStamp)

Решение?

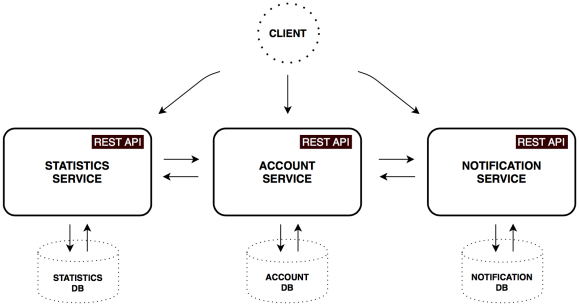
Сохранять логи в один log файл или в общую директорию

- Единая точка поиска
- Синхронизировать логи разных сервисов по времени (общий TimeStamp)

Проблемы

- Хранятся все логи беспорядочно
- Занимает место на диске
- Нужно управлять вручную

Как быть с микросервисами?



- Микросервисы часто не имеют доступ к общим ресурсам (низкий coupling).
- Нужен отдельный агент для каждого сервиса. Как им коммуницировать друг с другом?

Зачем нужен observability-стек

Observability

- Вся телеметрия в одном месте
- Сигналы из кода и инфраструктуры
- Данные сохраняются для анализа

Monitoring

Определение

Мониторинг — это непрерывное наблюдение за состоянием системы

Мониторинг может предупреждать только о заранее определённых проблемах

Зачем нужен?

- Вовремя замечать нестандартное поведение
- Быстро реагировать на инциденты

Observability

Observability — это способность понимать систему «снаружи», задавая вопросы без полного знания её внутреннего устройства.

Зачем нужен?

- Вся телеметрия в одном месте
- Хранит сигналы из кода и инфраструктуры
- Данные сохраняются для анализа

Observability знает контекст

Observability может показать

- Что произошло?
- Как произошло?
- Где произошло?

Monitoring vs Observability

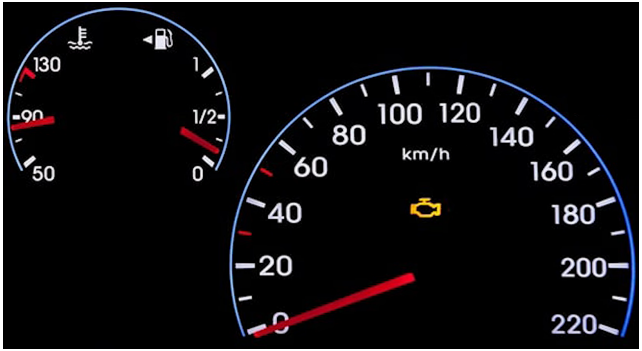
Monitoring

- Что?
- Когда?

Observability

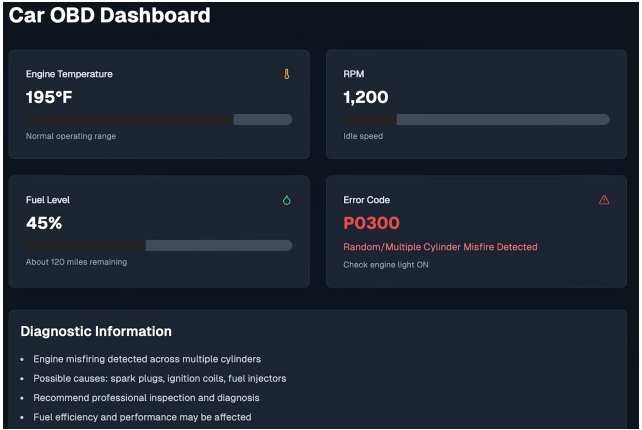
- Как?
- Почему?

Monitoring vs Observability



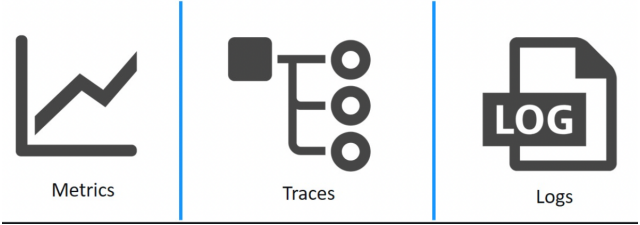
Monitoring

Monitoring vs Observability



Observability

Ключевые инструменты Observability



Телеметрия

- Metrics — агрегированные значения
- Logs — индивидуальные события в программе
- Traces — путь операции через всю систему

Metrics

Что это?

- числовые показатели во времени
- показывают нагрузку и качество работы
- хорошо подходят для графиков и алертов

Примеры

- latency, error rate, CPU, memory

Logs

Что это?

- текстовые записи о событиях в системе
- помогают понять локальный контекст ошибки
- полезны для диагностики конкретного случая

Примеры

- запуск операции, exception, ответ внешнего сервиса

Traces

Что это?

- цепочка шагов одного запроса
- показывает взаимодействие сервисов
- помогает находить узкое место и первопричину

Примеры

- gateway $\rightarrow$ auth $\rightarrow$ billing $\rightarrow$ database

Архитектура observability

Application $\rightarrow$ Collector $\rightarrow$ Storage $\rightarrow$ Visualization

Плохая архитектура observability

Application $\rightarrow$ Storage $\rightarrow$ Visualization

Почему так делать плохо?

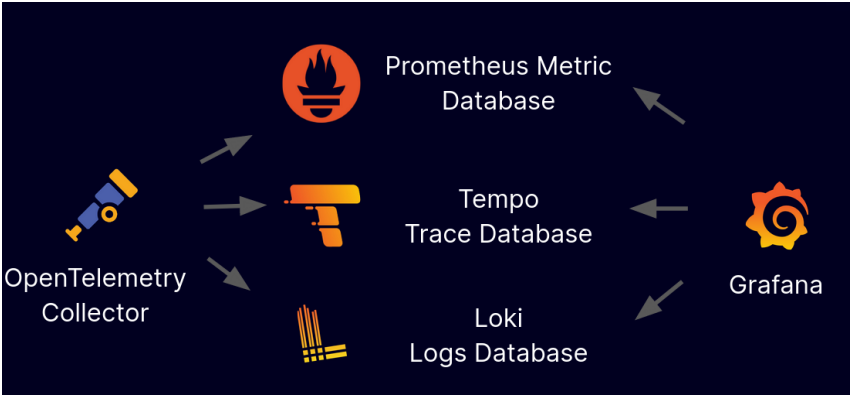
Плохая архитектура observability

$$\text{Application} \rightarrow \text{Storage} \rightarrow \text{Visualization}$$

Почему так делать плохо?

- Упало приложение -> упали логи
- Увеличивается coupling между сервисами

Сервисы для observability



Prometheus. Сбор метрик (metrics)

Что это?

- Система метрик и time series
- Сбор через HTTP запросы

Alloy. Сбор логов (logs) и путей (traces)

Что это?

- Alloy — агент для сбора и передачи телеметрии
- Собирает логи и трейсы и отправляет их в Loki и Tempo

Loki. Хранение логов (logs)

Что это?

- Система агрегации логов
- **Хранит** логи приложений и инфраструктуры

Подробнее про Loki

Ключевые моменты

- Агент отправляет строки логов в Loki
- Loki группирует их в потоки
- Поток определяется набором меток (labels)

Особенность Loki

Loki

- индексирует в основном labels
- текст логов хранит в чанках
- ниже стоимость хранения

Классический подход (Elasticsearch)

- индексируется полный текст
- поиск гибче по содержимому
- выше стоимость эксплуатации

Grafana. Визуализация

Что это?

- Grafana — инструмент для визуализации метрик, логов и трейсов
- Показывает данные на дашбордах и помогает анализировать систему

Источники информации и изображений

1. https://www.youtube.com/watch?v=ytx6jr2TyxI
2. https://dzone.com/articles/microservice-architecture-with-spring-cloud-and-do
3. https://medium.com/@nandanurseptamaa/instrumenting-microservices-with-open-telemetry-grafana-stack-fe154942cdff
4. https://grafana.com/docs/loki/latest/get-started/overview/
5. https://signoz.io/blog/prometheus-vs-influxdb/
6. https://hub.docker.com/r/grafana/alloy
7. https://medium.com/@surfd1001/things-to-know-about-observability-mechanisms-a52876e421c7

Photographs used under public domain or fair usage. No copyright infringement intended. If used accidently under wrong license please contact to be removed.

Программная инженерия

Лекция 10: Monitoring и Observability

Денисов Артём Владимирович

Уральский Федеральный Университет

2026